

Implementation of Convolutional Encoder and Viterbi Decoder using Verilog HDL

V.Kavinilavu¹, S. Salivahanan, V. S. Kanchana Bhaaskaran², Samiappa Sakthikumaran, B. Brindha and C. Vinoth
SSN College of Engineering, Kalavakkam, Rajiv Gandhi Salai, (Off) Chennai, Tamil Nadu, India
Email: vskanchana@hotmail.com

Abstract— A Viterbi decoder uses the Viterbi algorithm for decoding a bit stream that has been encoded using Forward error correction based on a Convolutional code. The maximum likelihood detection of a digital stream is possible by Viterbi algorithm. In this paper, we present a Convolutional encoder and Viterbi decoder with a constraint length of 7 and code rate of 1/2. This is realized using Verilog HDL. It is simulated and synthesized using Modelsim PE 10.0e and Xilinx 12.4i.

Keywords—Convolutional encoder, Viterbi decoder, Verilog HDL, Viterbi Algorithm.

I. INTRODUCTION

Convolutional coding is a coding scheme used in communication systems including deep space communications and wireless communications. It provides an alternative approach to block codes for transmission over a noisy channel. The block codes can be applied only for the block of data. The convolutional coding has an advantage over the block codes in that it can be applied to a continuous data stream as well as to blocks of data. IS-95, a wireless digital cellular standard for CDMA (Code Division Multiple Access), employs convolutional coding. A third generation wireless cellular standard, under preparation, plans to adopt turbo coding, which stems from the convolutional coding.

The Viterbi decoding algorithm was proposed and analyzed by Viterbi in 1967 [1]. It is widely used as a decoding technique for convolutional codes as well as the bit detection method in storage devices. Viterbi decoders currently find their use in more than one billion cell phones. The algorithm works by forming *trellis structure*, which is eventually traced back for decoding the received information. Convolutional encoding with Viterbi decoding is a powerful method for forward error correction. The Viterbi algorithm essentially performs maximum likelihood decoding. However, it reduces the computational complexity by using trellis structure. Figure 1 shows the convolutional encoder and Viterbi decoder, which is used in the digital communication system. Here, X is the input data stream, which is given into the convolutional encoder and it produces the encoded data stream (Y). The encoded data stream (Y) is given to the channel in the presence of noise. Hence, it produces the noise added encoded data stream (R). Finally, data stream (R) is given to the Viterbi decoder that produces the estimated data stream (Z) applied at the input.

II. CONVOLUTIONAL ENCODER

Convolutional codes are very suitable to protecting the digital data transmission from random errors due to any

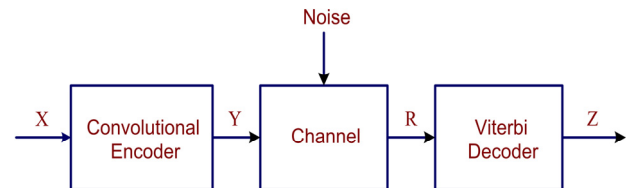


Figure 1. Convolutional encoder and Viterbi decoder

noise source. It achieves error free transmission by adding sufficient redundancy to the source symbols. Convolutional codes are usually described using two parameters: the code rate and the constraint length. The code rate is expressed as a ratio of number of input symbols (k) into the channel encoder to the number of output symbols (n) by the channel encoder in a given cycle. Then, the code rate is expressed as,

$$r = k/n \text{ bits/symbol}$$

The constraint length (K) denotes the length of the convolutional encoder. Convolutional encoder increases the length of the message sequence by adding redundant bits in order to increase the likelihood of detecting the transmitted sequence even if errors have occurred during transmission. Figure 2 shows the convolutional encoder of constraint length (K) = 7 and code rate (r) = 1/2. A convolutional encoder generates redundant bits by using modulo-2 convolutions. Hence, it is called as Convolutional encoder. If n modulo-2 adders are used, then it produces n outputs for each bit. The motivation of this paper is to realize a Viterbi

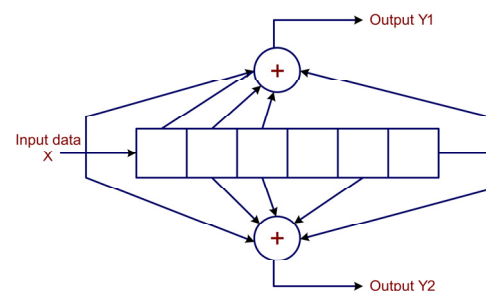


Figure 2. Constraint length (K)=7, code rate (r)=1/2 convolutional encoder.

decoder having Constraint length 7 and code rate 1/2 by using Modelsim 10.0e and Xilinx 12.4i tools.

Figure 3 shows the state information of a convolutional encoder. In the state diagram, the state information of the convolutional encoder is shown in circles. Then, it is stored in the shift registers. Each new input information bit causes a transition from one state to another. The path information between the states has been denoted as x/c . Here, x represents the input information bits and c represents the output encoded bits. It is customary to begin convolutional encoding from the *all zero* state. For example, the input information sequence $x=\{1011\}$ (begin from the all zero state) leads to the state transition sequence $s=\{10, 01, 10, 11\}$ and produces the output encoded sequence $c=\{11, 10, 00, 01\}$. Figure 3 shows the path taken through the state diagram for the given example.

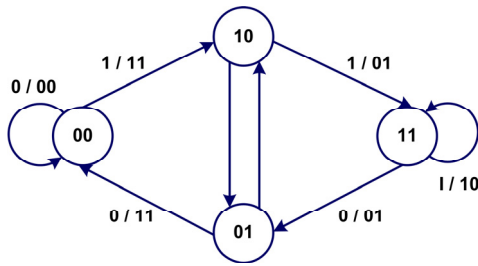


Figure 3. State diagram of Convolutional encoder

III. VITERBI ALGORITHM

The Viterbi algorithm has been known as a maximum likelihood decoding algorithm for convolutional codes. Let us consider a simple example for illustrating the principle of Viterbi algorithm. Assume that a car that has 3 states *forward*, *stop* and *reverse* with the condition that a transition from *forward* to *reverse* is not allowed. In other words, it implies that the car first enter the *stop* state and then enter the *reverse* state. Hence, when we receive the information through the processes of *forward*, *reverse* and *stop*, we can safely interpret it as *forward*, *stop* and *reverse* as this is a “maximum likelihood sequence”.

The Viterbi algorithm uses the trellis diagram to compute the path metric value (accumulated distance) from the received sequence to the possible transmitted sequences. The total number of such trellis paths increases exponentially with the number of stages in the trellis. It causes potential complexity and memory problems. The Viterbi decoding algorithm has been classified into hard decision decoding and soft decision decoding. If the received signal is converted into two levels, either zero or one, it is called hard decision. If the input signal is quantized and processed for more than two levels, it is called soft decision. The soft decision decoding is expensive and require large amount of memory than hard decision decoding. Hence, this work focuses on the hard decision decoding. Figure 4 shows the trellis diagram for hard decision Viterbi decoding.

When a sequence of data is received from the channel, it is desirable to estimate the original sequence that has been sent. The process of identifying such a sequence can be done using a diagram called ‘*trellis*’, which is shown in fig. 4. The

detection of the original stream can be described as finding the most probable path through the trellis.

In the trellis diagram each node specifies an individual

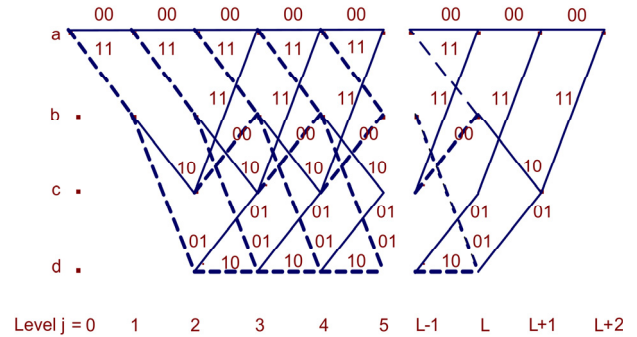


Figure 4. Hard decision Viterbi decoding trellis diagram

state at a given time and indicates a possible pattern of recently received data bits. The transition to a new state at the next timing cycle is indicated by each branch.

IV. VITERBI DECODER

Viterbi algorithm is used in the Viterbi decoder for decoding a bit stream that has been encoded using FEC based on a Convolutional code. Figure 5 shows the block diagram of Viterbi decoder [3]. It consists of the following functional units, namely, Branch Metric Unit, Path Metric Unit, Survivor Memory unit.

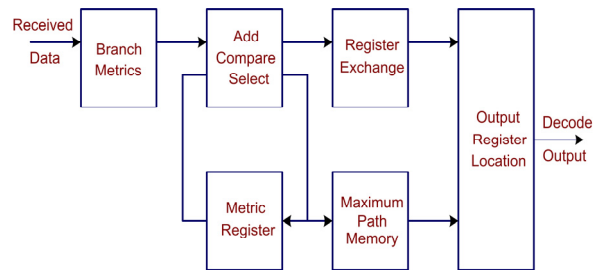


Figure 5. Block diagram of Viterbi decoder

A. Branch Metric Unit

The comparison between received code symbol and expected code symbol is done by branch metric unit. It also counts the number of differing bits. It is the smallest unit in the Viterbi decoder. The measured value of the BMU can be the Hamming distance in case of the hard input decoding or the Euclidean distance in case of the soft input decoding [4].

B. Path Metric Unit

The partial path metric of each branch is computed by the use of two adders. The partial path metric is compared by the comparator and an appropriate branch is selected by the selector. The selector selects the smaller value and stored

that value as the new path metric for each state. The corresponding bit decision is transferred to the SMU [4].

C. Survivor Memory Unit

The Survivor Memory Unit receives the bit decision from the PMU. This will produce the decoded sequence. After finishing the SMU, it is important to perform the trace back module. When the trellis diagram is finished, the trace-back module will search the maximum likelihood path from the final state which is state0 to the beginning state which is state0. Each time, the trace-back block just left shifts one bit of the binary state number and add one bit from the survivor path metric to compute the previous state. It stores the path information in the form of an array of recursive pointers. By doing this, the most likelihood path is found.

V. IMPLEMENTATION

The Convolutional encoder and Viterbi decoder are implemented using Verilog HDL and the code has been developed under full-custom design. This implementation is complicated when using Verilog HDL compared to VHDL [2]. The Convolutional encoder encodes the given input sequence by modulo-2 adders. The operation is shown in fig. 2.

The Viterbi decoder decodes the original input sequence by using Viterbi algorithm. The design flow of the Viterbi decoder is shown in fig. 6.

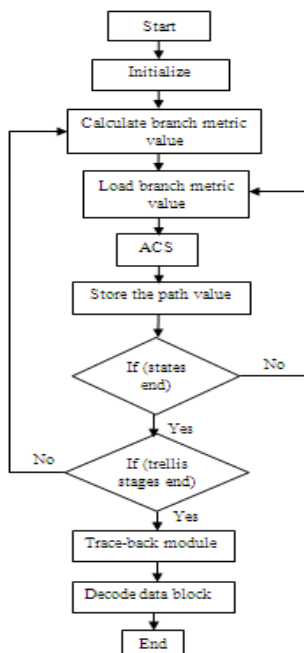


Figure 6. Design flow of Viterbi decoder

From the trellis structure, we calculate the branch metric value and load that value. These values are added and compared. Then, the smallest value found in the process is the new path metric value. If all the states and the trellis stages are finished, then the trace back module is performed. It produces the decoded data (original input data).

TABLE I. DEVICE UTILIZATION SUMMARY OF THE CONVOLUTIONAL ENCODER

Design Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slices	325	2448	13%
Number of Slice Flip Flops	23	4896	0%
Number of 4 input LUTs	602	4896	12%
Number of bonded IOBs	39	66	59%
Number of GCLKs	1	24	4%

VI. RESULTS AND DISCUSSIONS

The Convolutional encoder for the constraint length of $K=7$ and code rate of $r=1/2$ has been developed and the synthesis is carried out. Table I shows the device utilization summary of the convolutional encoder. It has been simulated and the simulation result is shown in fig. 7.

Typical input and output are as indicated below.

Input bits $X = [11101100]$;

Encoded bits $Y = [1101100100010111]$;

Encoded noise output $R = [1101111100010111]$.

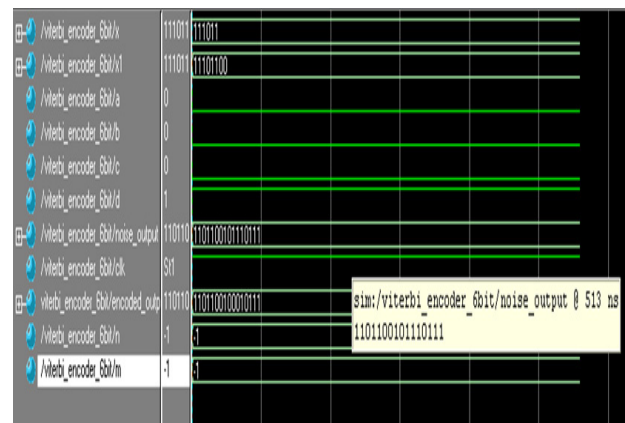


Figure 7. Simulation result of Convolutional encoder

The Viterbi decoder has been developed using and the synthesis is carried out. Table II shows the device utilization summary of the Viterbi decoder. It has been simulated and the simulation result is shown in fig. 8.

TABLE II. DEVICE UTILIZATION SUMMARY OF THE VITERBI DECODER

Design Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slices	39	2448	1%
Number of Slice Flip Flops	36	4896	0%
Number of 4 input LUTs	78	4896	1%
Number of bonded IOBs	27	66	4%
Number of GCLKs	1	24	4%

Typical input and output are as indicated below.

Encoded noise input bits $R = [1101111100010111]$;
Decoded output bits $Z = [11101100]$.

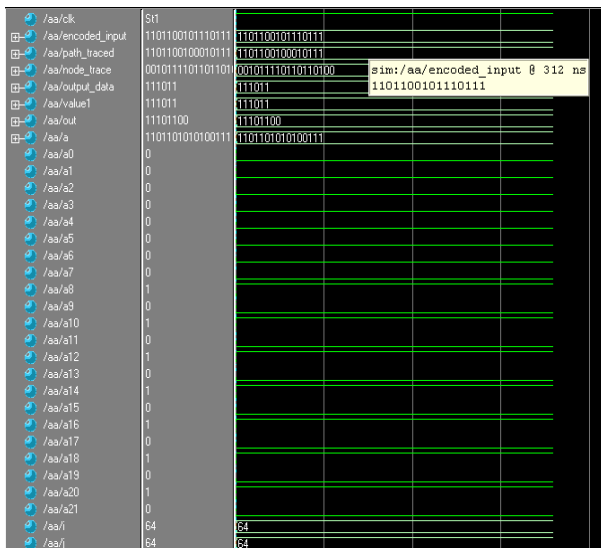


Figure 8. Simulation result of Viterbi decoder

VII. CONCLUSION

In this paper, we have presented the design and implementation of the Convolutional encoder and Viterbi decoder. This design has been simulated in MODELSIM 10.0e and synthesized using XILINX-ISE 12.4i for the constraint length of $K=7$ and code rate of $\frac{1}{2}$ input sequence. The given input sequence has been encoded by using convolutional encoder and it is transmitted through the channel. Finally, the transmitted sequence is decoded by the Viterbi decoder and the estimated original sequence is produced.

REFERENCES

- [1] A.J.Viterbi, "Error bounds for convolutional coding and an asymptotically optimum decoding algorithm", IEEE Tran. on Inform. Theory, Vol. 2, Pp. 260-269, Apr. 1967.
- [2] Wong, Y.S. et.al "Implementation of convolutional encoder and Viterbi decoder using VHDL" IEEE Tran. on Inform. Theory, Pp. 22-25, Nov. 2009.
- [3] Hema.S, Suresh Babu.V and Ramesh.P, "FPGA Implementation of Viterbi decoder" proceedings of the 6th WSEAS ICEHWOC, Feb. 2007.
- [4] İrfan Habib, Özgün Paker, Sergei Sawitzki, "Design Space Exploration of Hard-Decision Viterbi Decoding: Algorithm and VLSI Implementation" IEEE Tran. on Very Large Scale Integration (VLSI) Systems, Vol. 18, Pp. 794-807, May 2010.